



CAPACITA
iRede

TAREFA

Unidade 3 | Capítulo 1

Desenvolvimento de Projeto

Prof.: Allberson Dantas



Unidade 3

Capítulo 1 | Objetivo 6

Enunciado:

Com o desenvolvimento do projeto os alunos colocam em prática todos os conteúdos abordados no módulo inicial, aplicando de forma autônoma os conhecimentos adquiridos sobre Java, orientação a objetos, estruturas de controle, arrays, tratamento de exceções e organização de código.

Instruções:

A turma deve escolher um dos projetos abaixo e seguir todas as instruções correspondentes ao projeto selecionado. Caso o aluno opte por desenvolver um projeto diferente dos sugeridos, deverá informar ao professor, em detalhes, como será a proposta. Ressalta-se que o projeto escolhido será desenvolvido de forma incremental ao longo do curso e dos módulos, com 3 (três) entregas previstas, sendo 1 (uma) em cada módulo.

Unidade 3

Capítulo 1

Projeto 1: TaskManager – Gerenciador de Tarefas (Versão Console)

Objetivo do Projeto:

O objetivo deste projeto é desenvolver, passo a passo, um Sistema de Gerenciamento de Tarefas simples em Java, rodando no console (linha de comando). O sistema será criado com Java puro, aplicando os conceitos fundamentais de Programação Orientada a Objetos (POO) aprendidos no módulo.

Este projeto servirá como uma base sólida para evoluções futuras, como uma interface gráfica com JavaFX e, posteriormente, uma API web com Spring Boot.

Descrição Geral:

O sistema TaskManager permitirá ao usuário realizar operações básicas de controle de tarefas. Nesta versão inicial, ele poderá:

- Criar novas tarefas
- Listar todas as tarefas
- Marcar tarefas como concluídas
- Remover tarefas existentes

A interação será feita via terminal (console) e todas as informações estarão em memória (não há persistência em banco nesta etapa).

Os alunos aprenderão os conceitos fundamentais da linguagem Java. Para o projeto, os seguintes tópicos serão usados:

Fundamentos Utilizados:

- Scanner para entrada de dados
- System.out.println para saída no console
- Estruturas de decisão: if, else, switch
- Laços de repetição: for, while, do-while
- Manipulação de listas com ArrayList<Tarefa>
- Manipulação básica de String (formatação, comparação, divisão com split, etc.)

Os alunos aprenderão os pilares da POO e os aplicarão no projeto:

- Classes e Objetos:

Criação da classe Tarefa com os atributos:

```
1 String titulo;  
2 String descricao;  
3 boolean concluida;
```


Lógica do Sistema:

- Criar a classe TaskManager, que encapsula a lógica e armazena as tarefas em um ArrayList<Tarefa>
- Métodos principais do TaskManager:
 - adicionarTarefa()
 - listarTarefas()
 - concluirTarefa()
 - removerTarefa()

Conceitos de POO Aplicados:

- Encapsulamento com private e uso de getters/setters
- Herança: exemplo com uma classe TarefaPrioritaria que herda de Tarefa
- Polimorfismo: tratamento genérico de tarefas e subtarefas
- Classe abstrata BaseTarefa (opcional), ou interface ITarefa
- Tratamento básico de exceções com try/catch para entradas inválidas

Organização do Projeto em Pacotes:

— model/	→ classes como Tarefa, TarefaPrioritaria
— controller/	→ TaskManager (lógica de gerenciamento)
— app/	→ classe principal com o menu interativo
— exceptions/	→ customização de exceções (opcional)

Na etapa final, os alunos irão construir uma interface de texto (console) com um menu básico que permite a interação do usuário com o sistema:

Exemplo de Menu no Console:

===== GERENCIADOR DE TAREFAS =====

1. Criar nova tarefa
2. Listar tarefas
3. Marcar tarefa como concluída
4. Remover tarefa
5. Sair

Escolha uma opção:

O menu deve funcionar em loop:

Após executar uma ação, o menu aparece novamente até que o usuário selecione "Sair".

Projeto 2: Finanças Pessoais (Versão Console)

Nome do Projeto: FinTrack

Descrição do Projeto (Versão Inicial)

O FinTrack é um sistema de controle de finanças pessoais via console, permitindo ao usuário:

- Cadastrar entradas (receitas) e saídas (despesas)
- Listar todas as transações
- Exibir saldo atual
- Remover uma transação

O projeto será feito com Java puro, com foco na prática de POO e estruturas básicas, e poderá ser expandido futuramente com relatórios gráficos, persistência em banco de dados e integração com APIs.

Funcionalidades no Módulo Inicial

- Uso de Scanner, System.out
- Condições com if, switch
- Laços de repetição com for, while
- Uso de ArrayList
- Manipulação de String e double

Programação Orientada a Objetos:
Classe Transacao com:

```
1  String descricao;  
2  double valor;  
3  boolean ehReceita;  
4  LocalDate data;  
5
```

Classe FinTracker com métodos:

adicionarTransacao()
listarTransacoes()
removerTransacao()
calcularSaldoTotal()

Aplicação de:

- Encapsulamento com getters/setters
- Polimorfismo (ex: Transacao e TransacaoMensal)
- Tratamento de exceções com try/catch

Organização em pacotes: model, controller, app, exceptions

fintrack/

├── app/

└─ Main.java

→ Classe principal com o menu e execução (Não precisa ser nomeado Main.java!)

├── controller/

└─ FinTracker.java

→ Lógica principal para gerenciar as transações

├── model/

└─ Transacao.java

→ Classe base para qualquer tipo de transação

└─ TransacaoMensal.java

→ Subclasse que representa uma transação recorrente (ex: salário fixo)

├── exceptions/

└─ EntradaInvalidaException.java

→ Exceções personalizadas (opcional)

├── utils/

└─ Formatador.java

→ Funções auxiliares para formatar datas, valores, etc. (opcional)

Exemplo de Menu:

===== FINTRACK - SEU CONTROLE FINANCEIRO =====

1. Adicionar nova transação
2. Listar transações
3. Mostrar saldo atual
4. Remover transação
5. Sair

O menu deve funcionar em loop:

Após executar uma ação, o menu aparece novamente até que o usuário selecione "Sair".

Projeto 3: Agenda de Contatos (Versão Console)

Nome do Projeto: MyContacts

Descrição do Projeto (Versão Inicial):

O MyContacts é um sistema simples de agenda de contatos no console. O usuário poderá:

- Adicionar contatos
- Listar contatos
- Buscar contatos por nome
- Remover contatos

Este projeto reforça o uso de POO, manipulação de listas e pesquisa de dados simples.

Funcionalidades no Módulo Inicial:

- Scanner para entrada
- Condições e laços de repetição
- Uso de ArrayList e String
- Comparações com equalsIgnoreCase

Programação Orientada a Objetos:

Classe Contato com:

```
1  String nome;  
2  String telefone;  
3  String email;
```

Classe Agenda com:

- adicionarContato()
- listarContatos()
- buscarPorNome()
- removerContato()

Conceitos aplicados:

- Encapsulamento
- Interface Buscavel (opcional)
- Herança: ContatoComercial com campo empresa
- Tratamento de exceções simples

Organização modular com pacotes:

mycontacts/
└─ app/

└ Main.java	→ Menu e interação do usuário (Não é obrigatório nomear Main.java)
└─ controller/ └ Agenda.java	→ Lógica de manipulação da lista de contatos
└─ model/ └ Contato.java	→ Classe base com nome, telefone, email
└ ContatoComercial.java	→ Subclasse com campo adicional (empresa)
└─ exceptions/ └ ContatoNaoEncontradoException.java	→ Exceção customizada
└─ utils/ └ ValidadorEmail.java	→ Classe para validar formato de email (opcional)

Exemplo de Menu:

===== AGENDA DE CONTATOS =====

1. Adicionar novo contato
2. Listar contatos
3. Buscar por nome
4. Remover contato
5. Sair

O menu deve funcionar em loop:

Após executar uma ação, o menu aparece novamente até que o usuário selecione "Sair".



CAPACITA iRede

RESIDÊNCIA EM TIC 20

EXECUTOR



INICIATIVA



COORDENADORA PPI



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.